Lab Task Report

Only for course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	
Clarity						
Content Quality						
Spelling & Grammar						
Organized on and Forma ng						
Total obtained mark						
Comments						

Semester: January-June 2026

Student Name: Protul Devnath

Student ID: 0022520005101o89

Batch: 45th Sec on: C

Course Code: 124 Course Name: Object Oriented Programming

Course Teacher Name: Debabarata Mallick

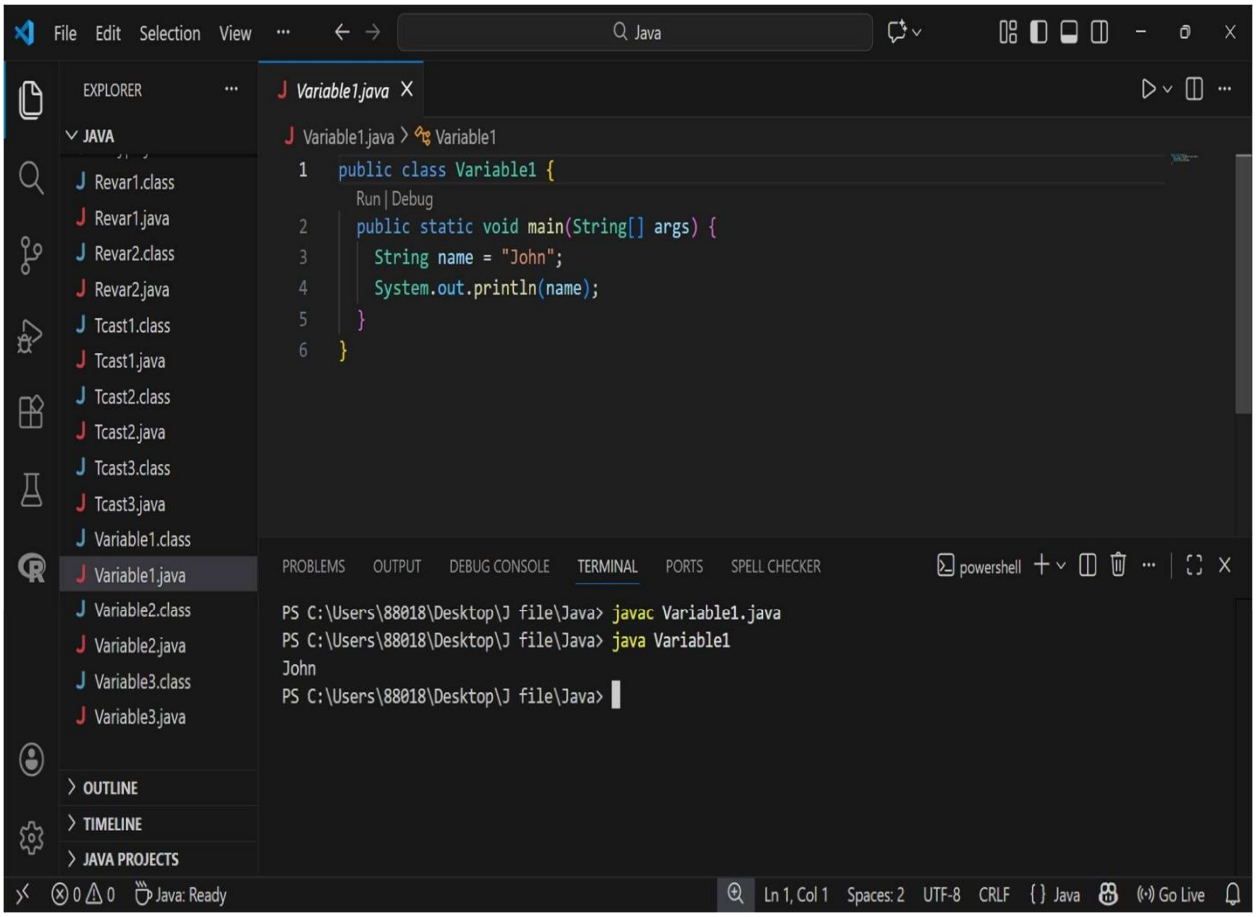
Designation: Lecturer

Submission Date: 23/01/26

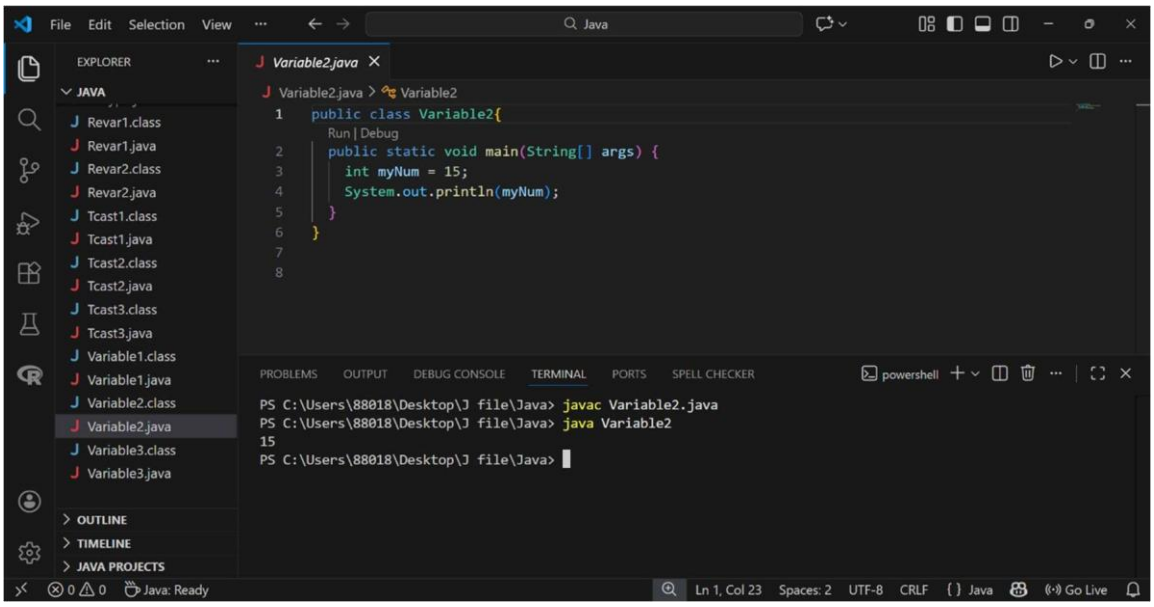
GitHub-https://github.com/Protul518

OOP TASK-3 Variables sec on:

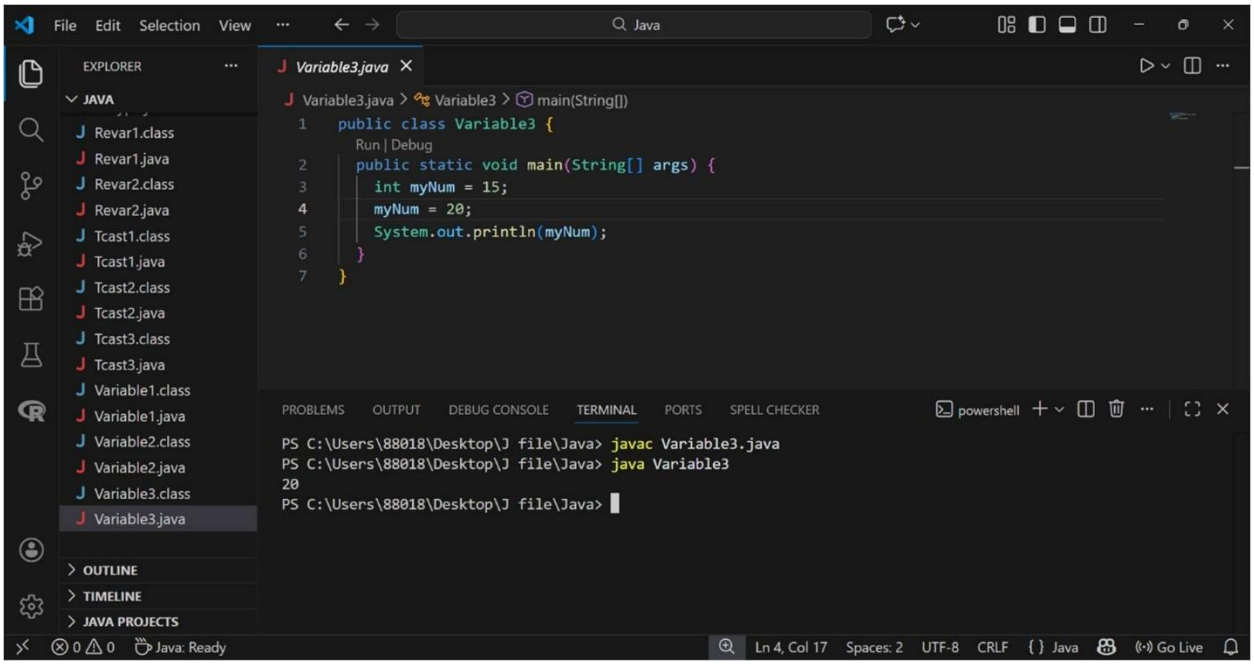
1. This Java program demonstrates variable declara on and ini aliza on by assigning the string literal "John" to a String variable named name. The program then u lizes the System.out.println() method to output the value of the variable to the console, successfully displaying "John" upon execu on.



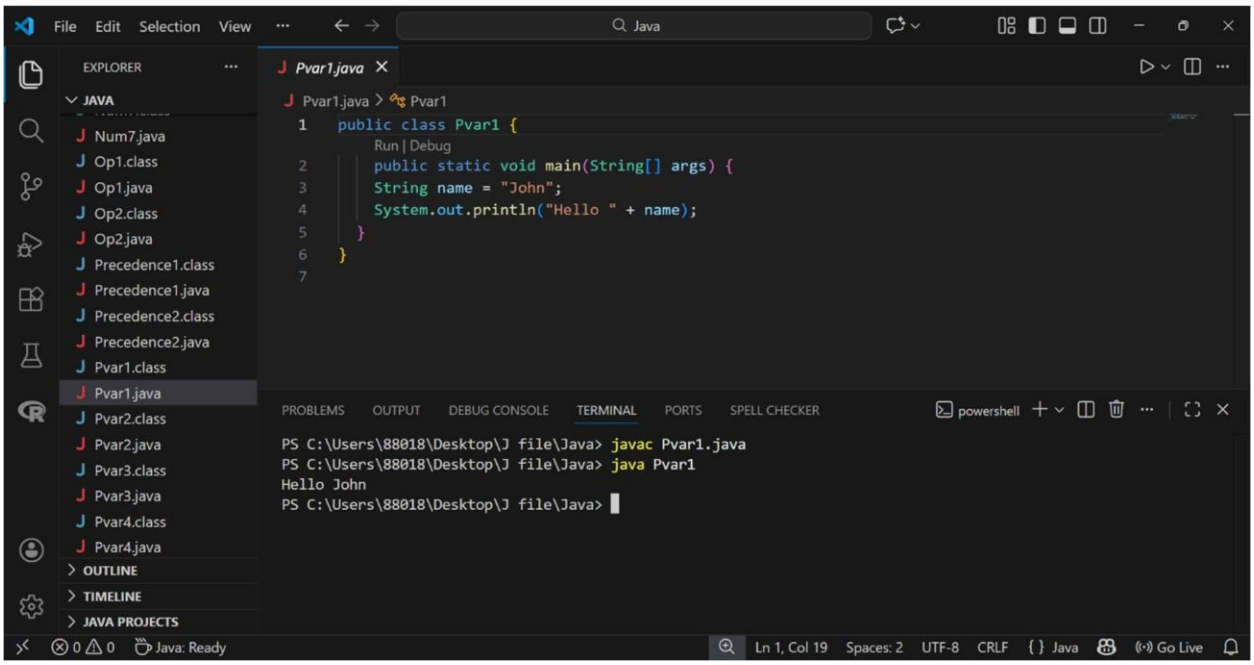
2. This Java program illustrates the declara on and ini aliza on of an integer variable, assigning the numerical value of 15 to the iden fier myNum. The code subsequently invokes the System.out.println() func on to print the stored integer value to the terminal, as confirmed by the program output in the terminal window.



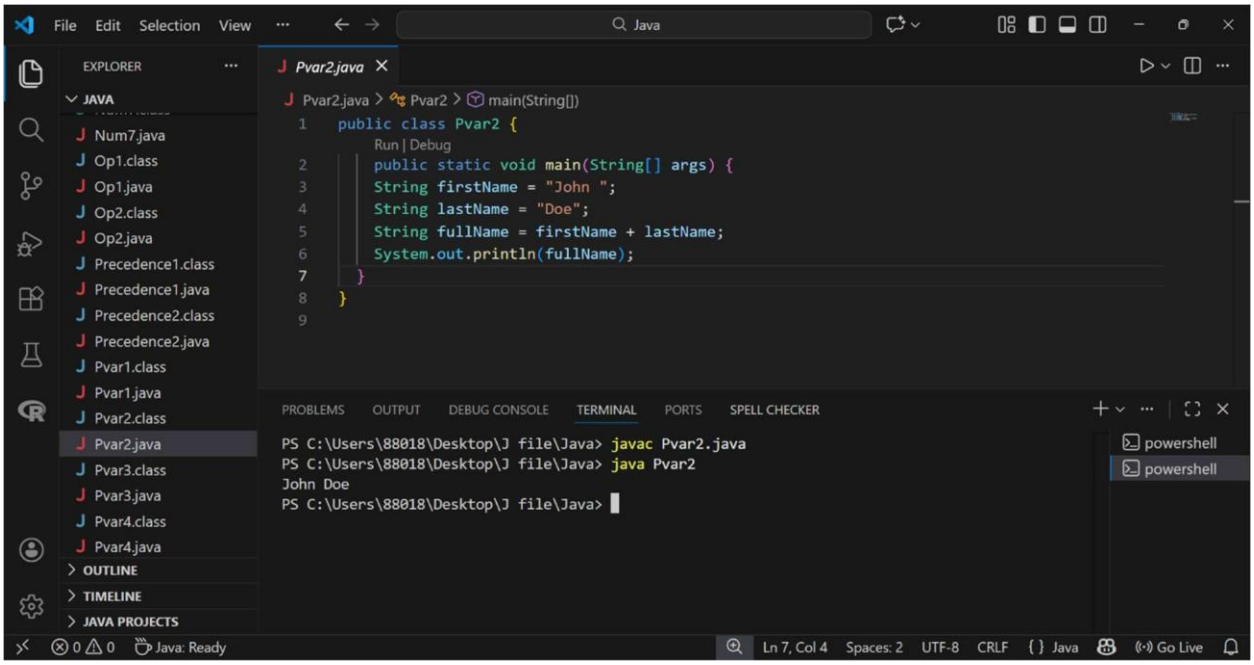
3. This Java program demonstrates variable reassignment, where an integer variable myNum is ini ally declared with the value 15 and subsequently updated to 20. The final value is then printed to the console using System.out.println(), illustra ng that variables can hold different values over me during program execu on.



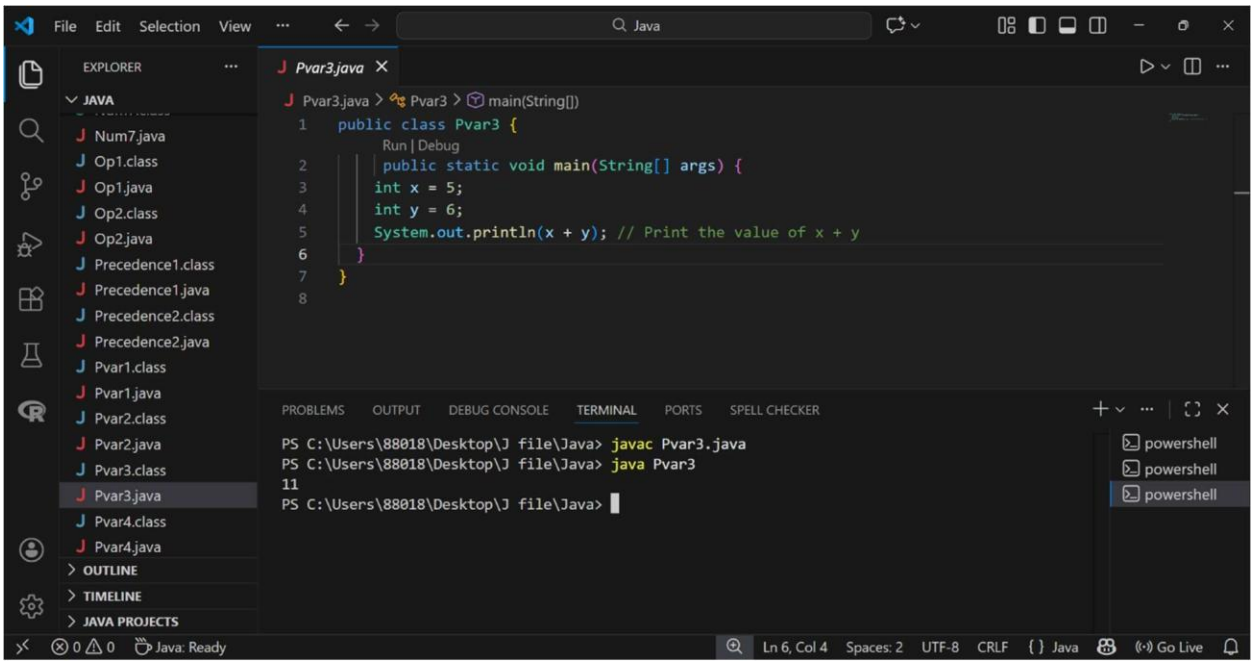
4. This program highlights string concatena on by using the + operator to combine the string literal "Hello " with the value stored in the variable name. The resul ng combined string, "Hello John", is then printed to the terminal, showing how sta c text can be dynamically joined with variable data.



5. This program demonstrates complex string manipula on by combining two separate string variables, firstName and lastName, into a new variable called fullName for display.

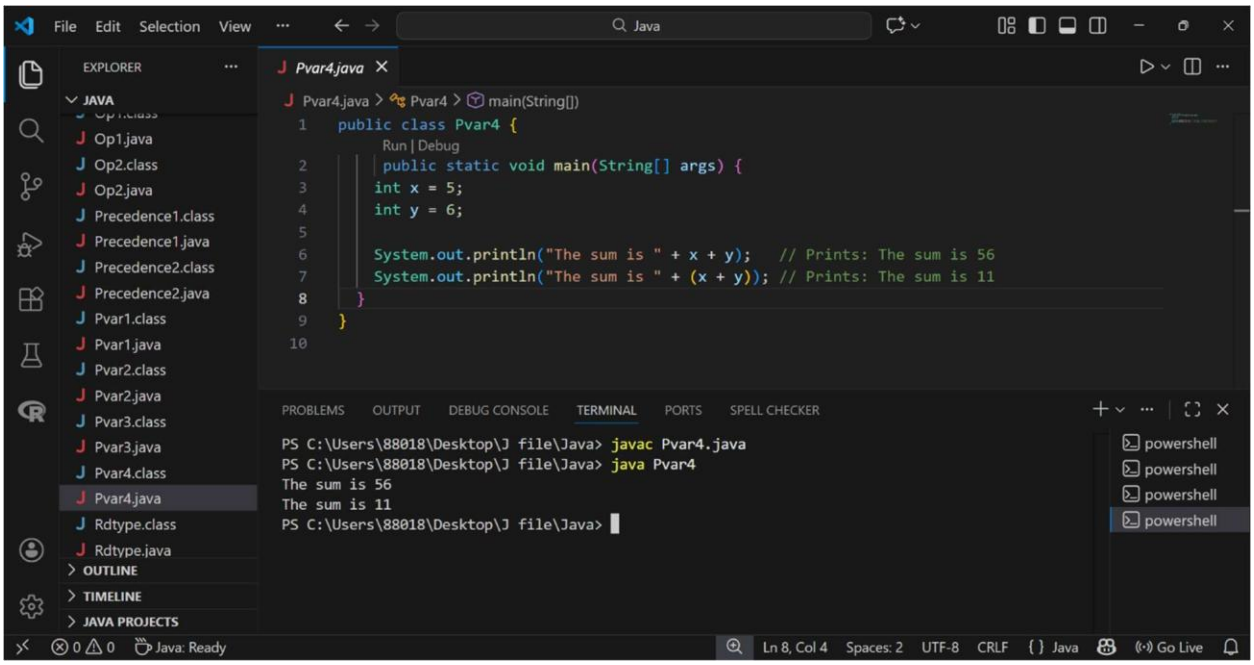


6. This code performs arithme c addi on by using the + operator to calculate and print the sum of two integer variables, x and y.

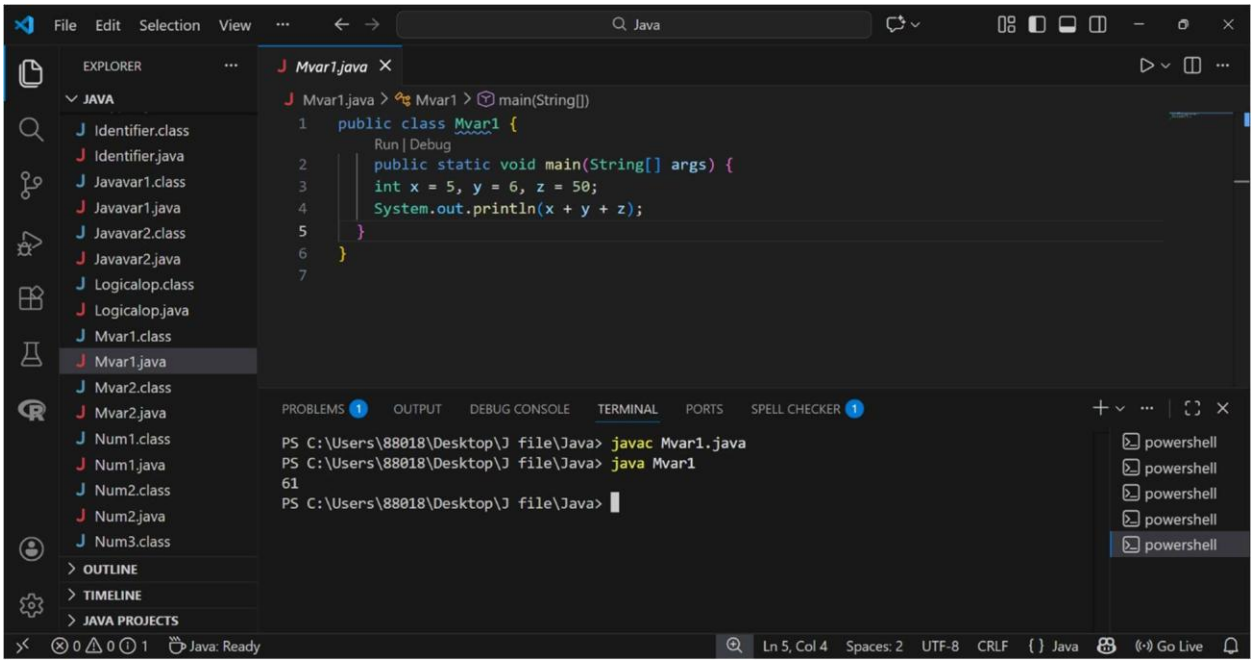


7. This program highlights the impact of operator precedence by showing how parentheses can force mathema cal addi on (result 11) rather than default string concatena on

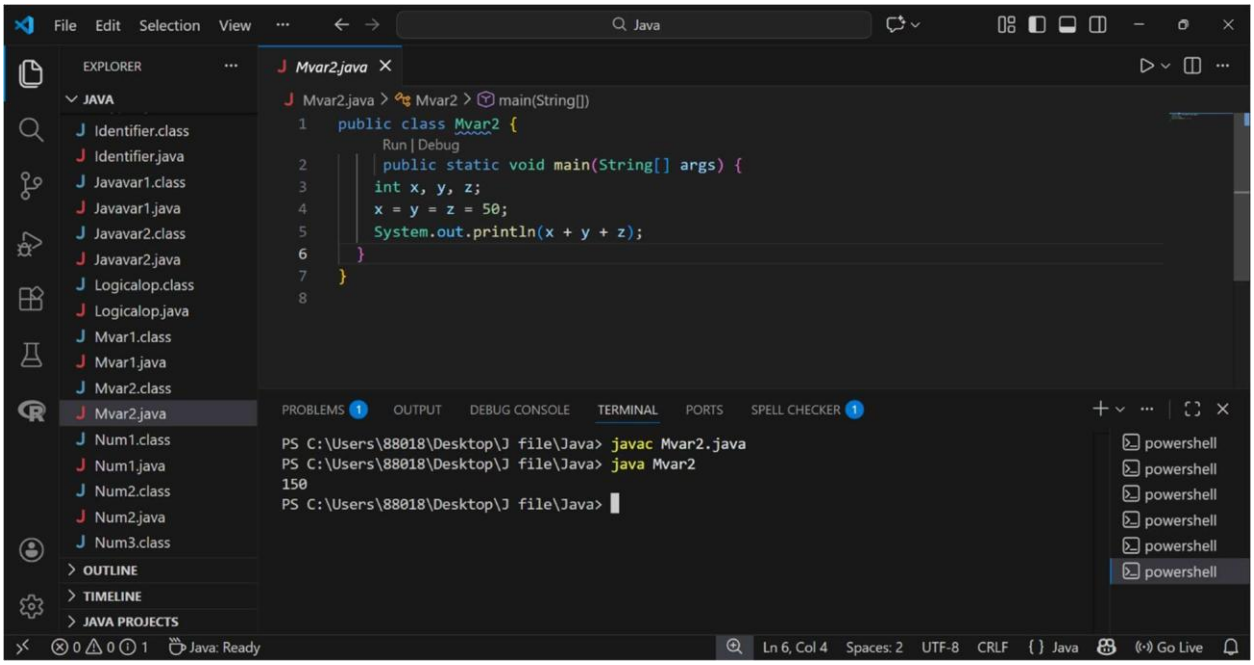
(result 56).



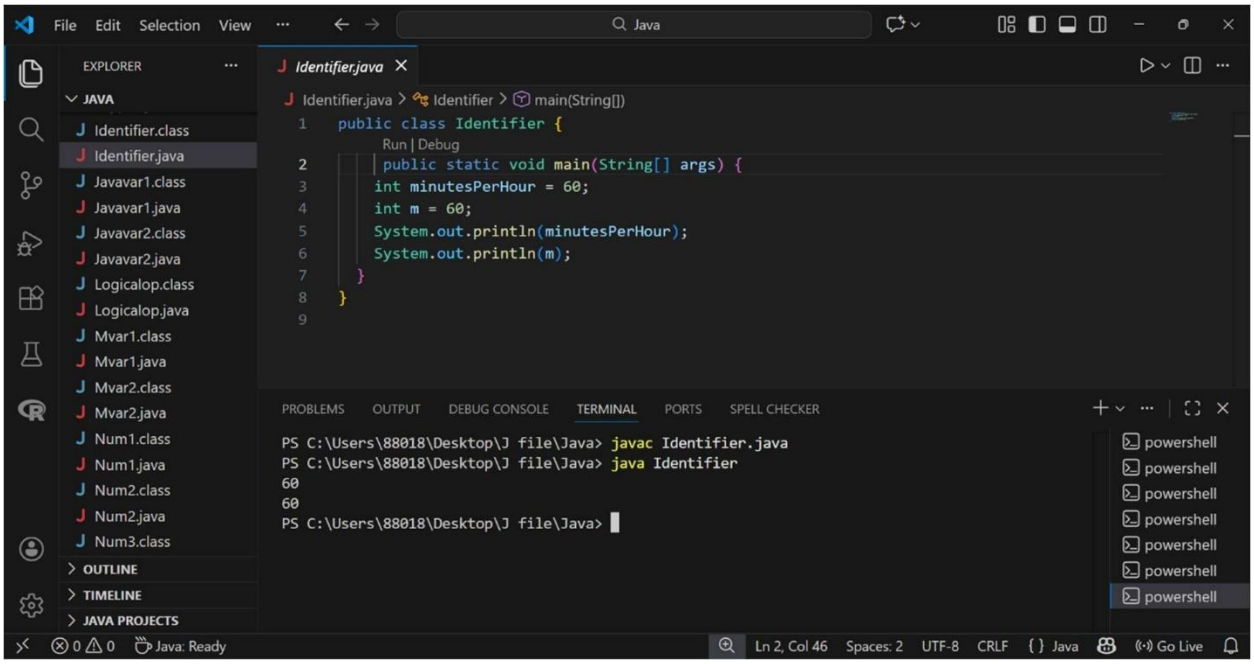
8. This code demonstrates mul ple variable declara on on a single line, ini alizing three separate integers (x, y, and z) and prin ng their collec ve sum.



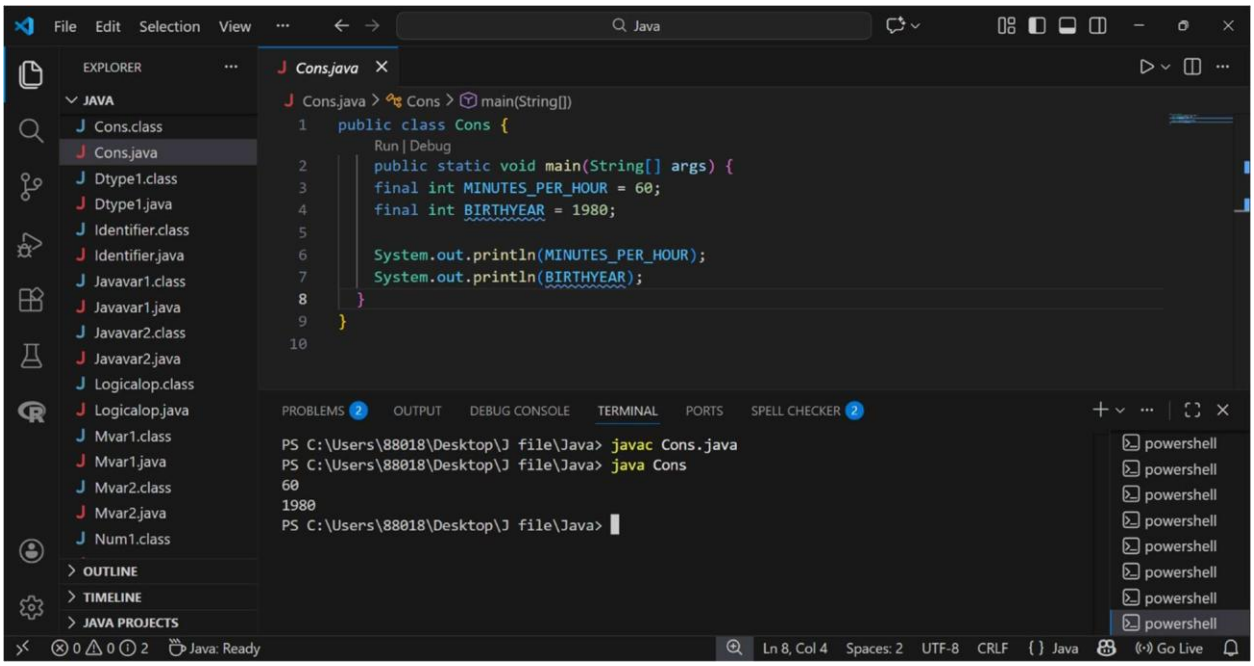
9. This script illustrates mul ple variable assignment to a common value, where three previously declared integers are all set to 50 in a single statement.



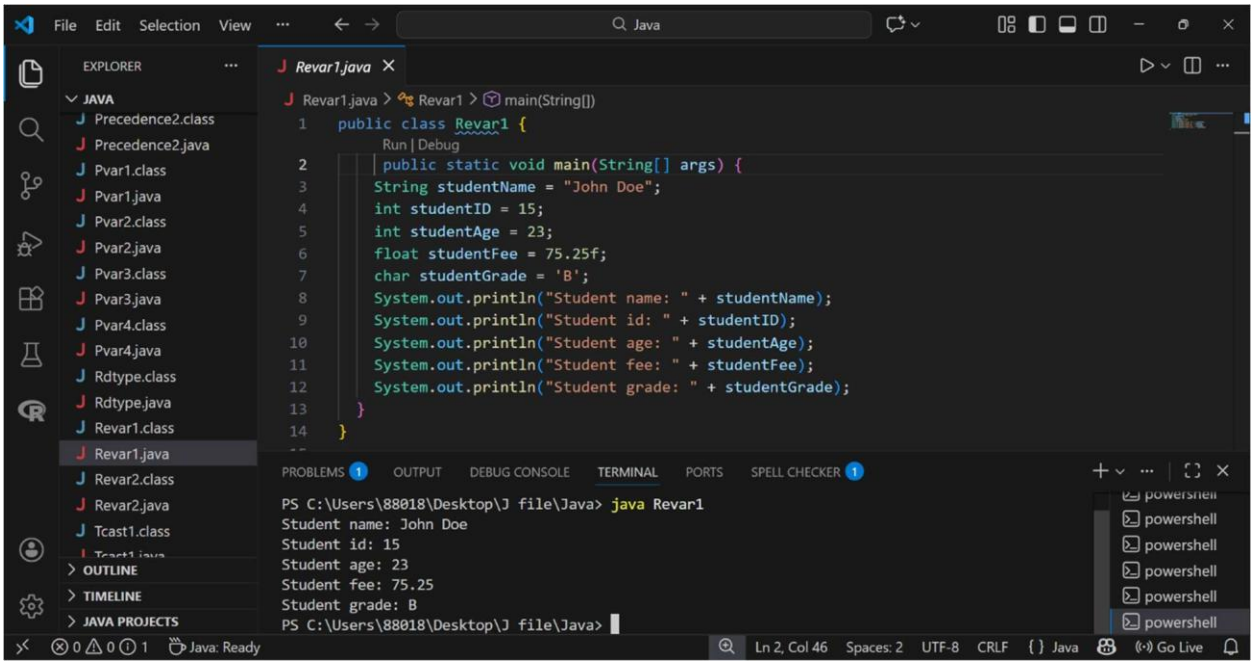
10. This program illustrates the importance of meaningful iden fiers, comparing the descrip ve minutesPerHour against the vague variable name m.



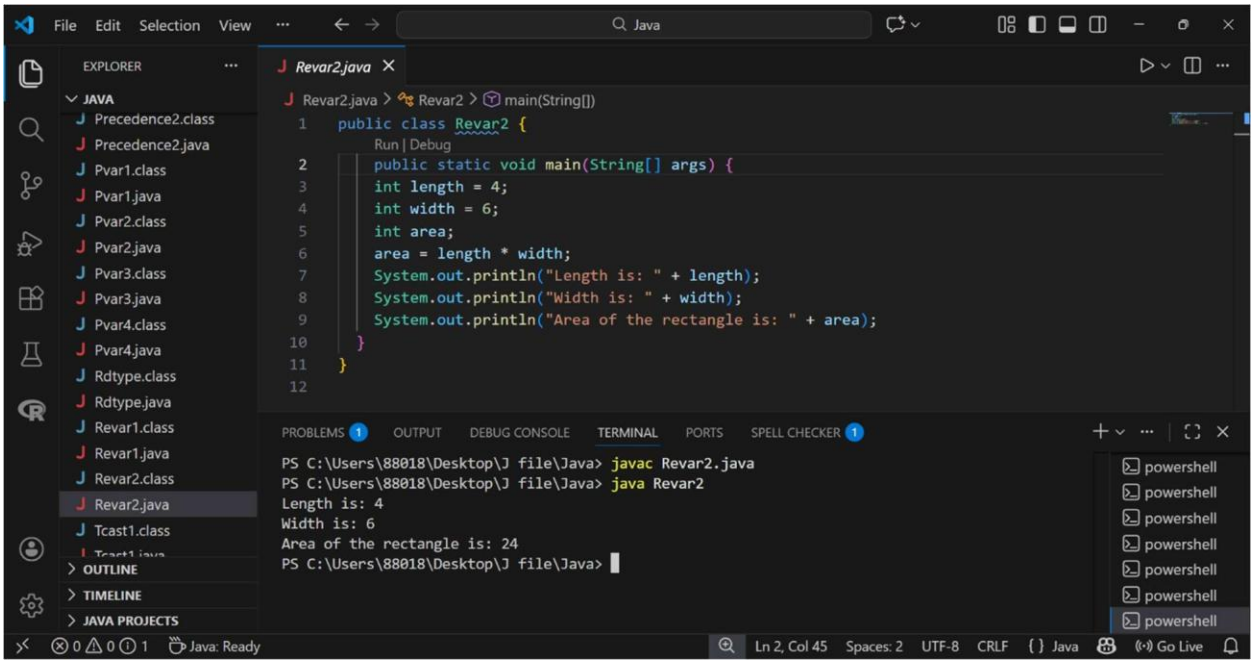
11. This script demonstrates the use of the final keyword to declare constants, ensuring that the values of MINUTES_PER_HOUR and BIRTHYEAR cannot be modified during execu on.



12. This program provides a comprehensive example of diverse data type usage, declaring and displaying String, int, float, and char variables in a single report-style output.

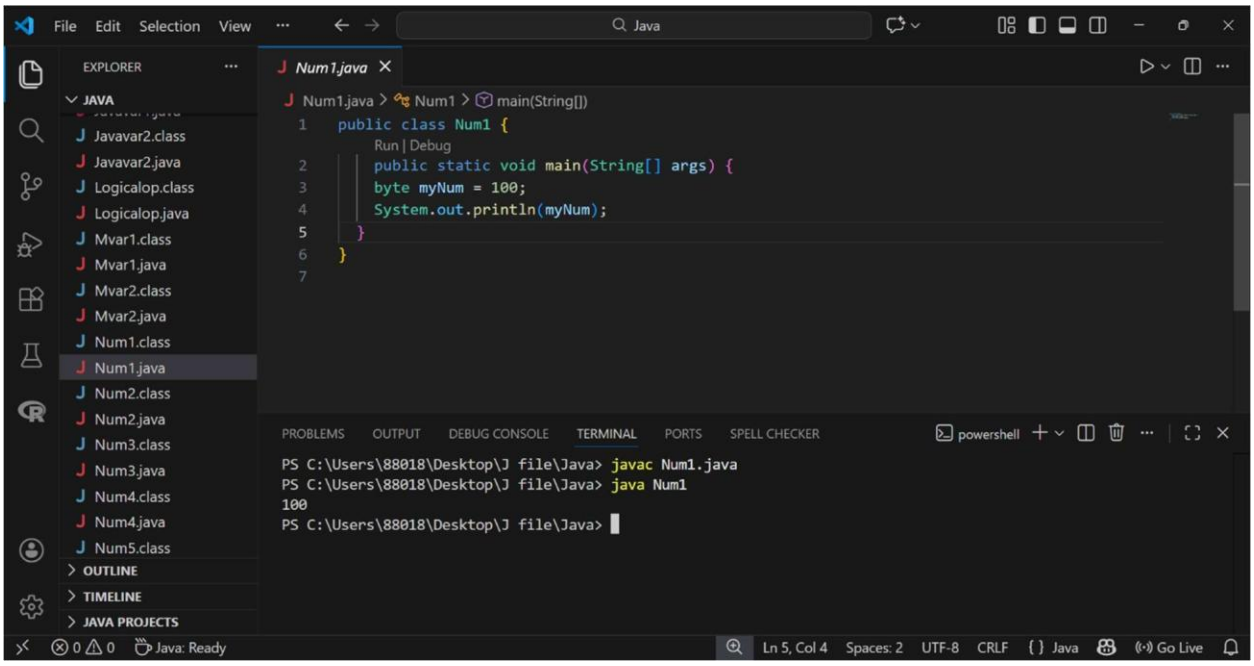


13. This code illustrates a prac cal applica on of variables by using integers to calculate and display the area of a rectangle through a mathema cal formula.

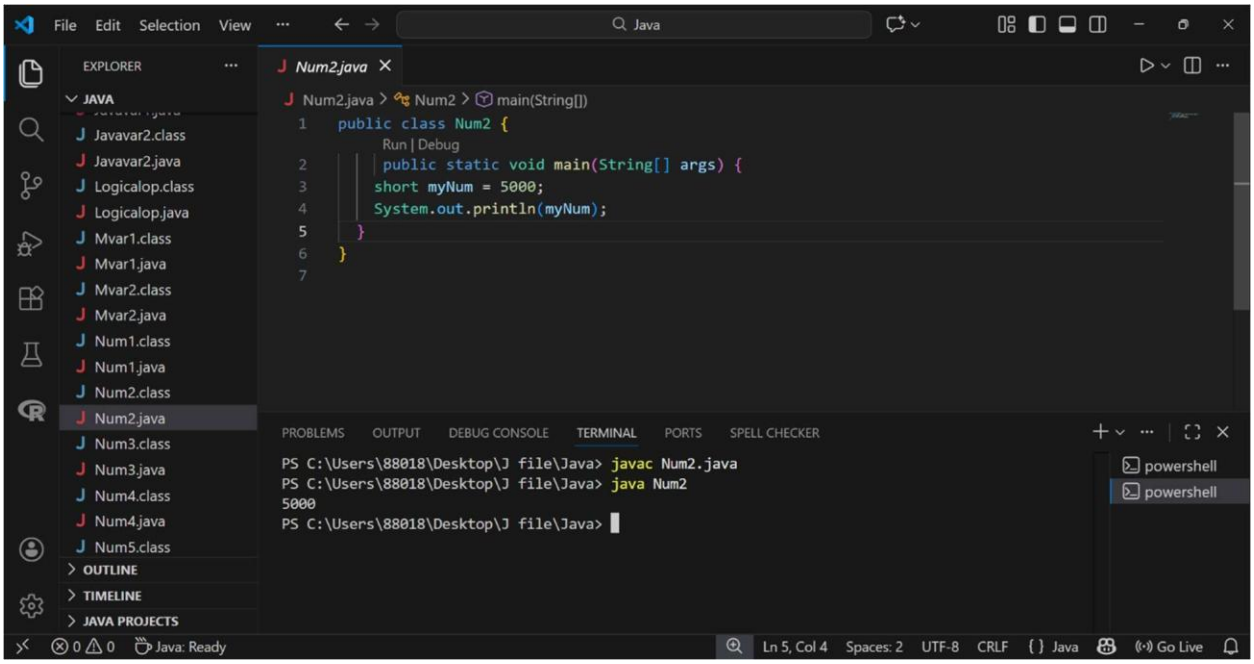


Data Types sec on:

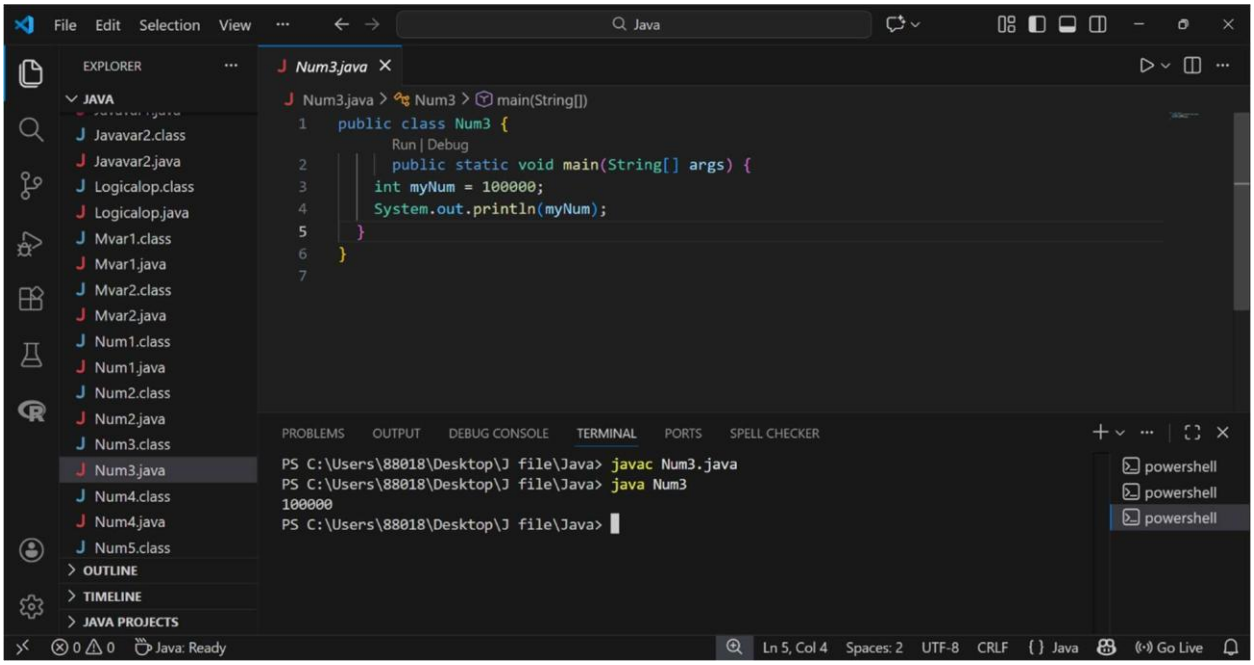
1. The provided Java code demonstrates the declara on of a byte data type variable named myNum, assigns it a value of 100, and then outputs that value to the console using the System.out.println() method.



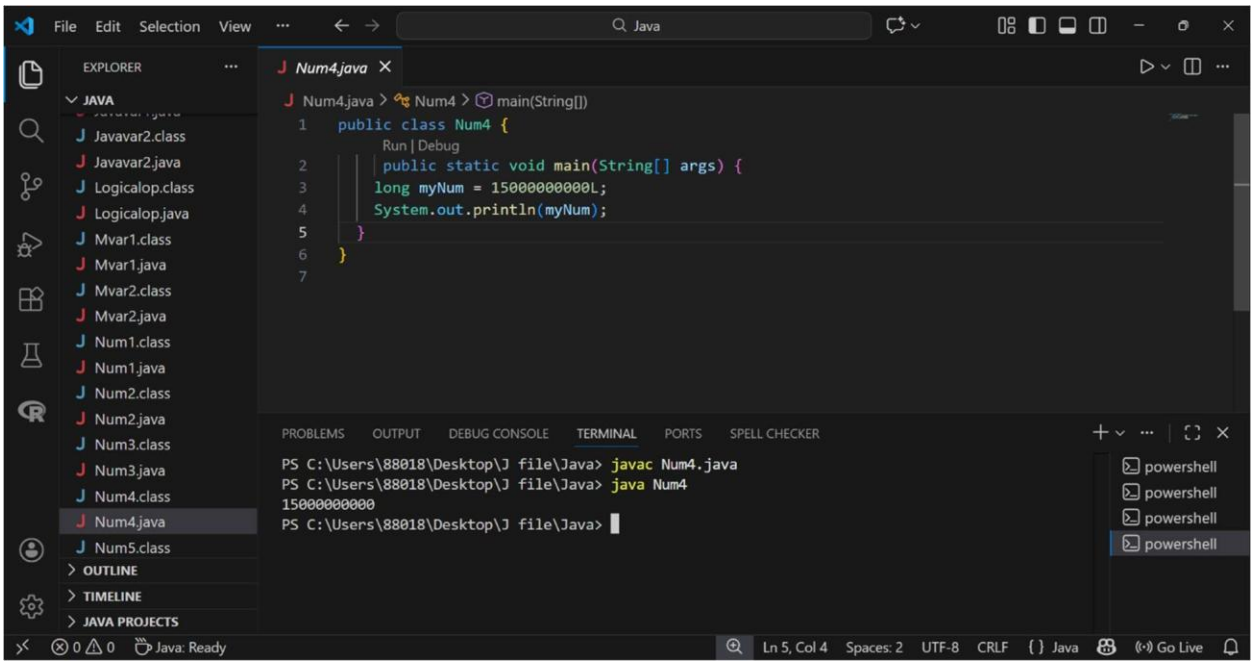
2. This Java program illustrates the use of the short primi ve data type to store a 16-bit integer value of 5000 in the variable myNum before prin ng it to the terminal.



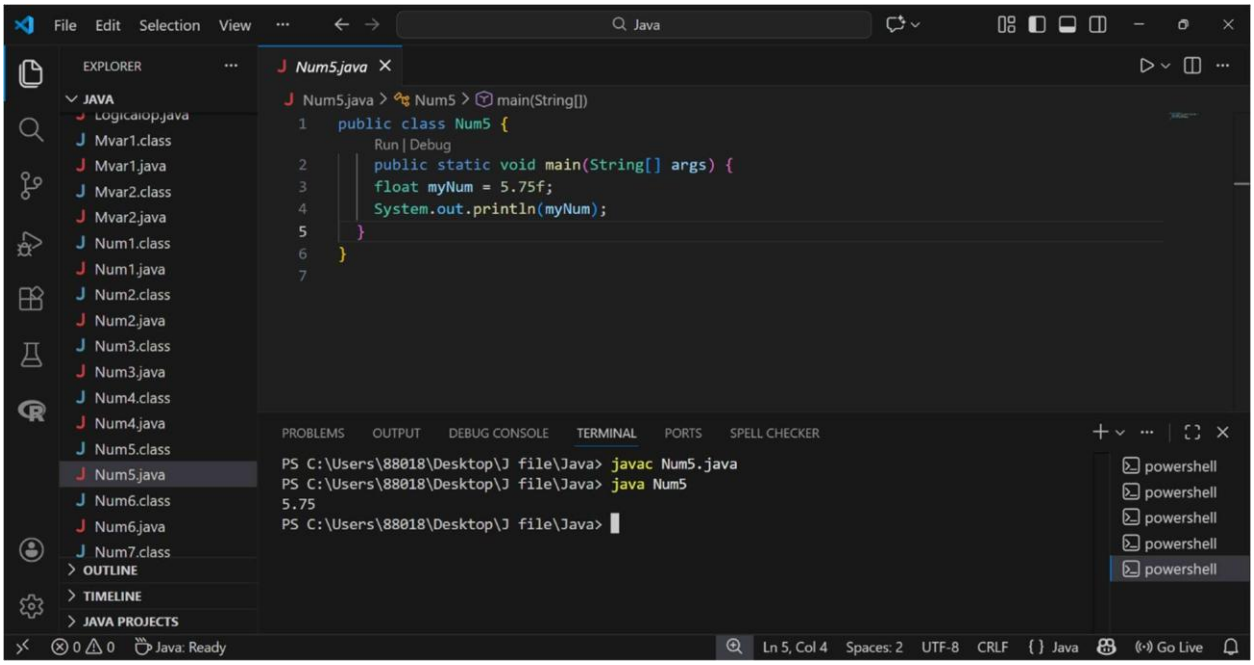
3. This Java program demonstrates the use of the int data type to store a large 32-bit integer value of 100000 in the variable myNum and displays it in the terminal.



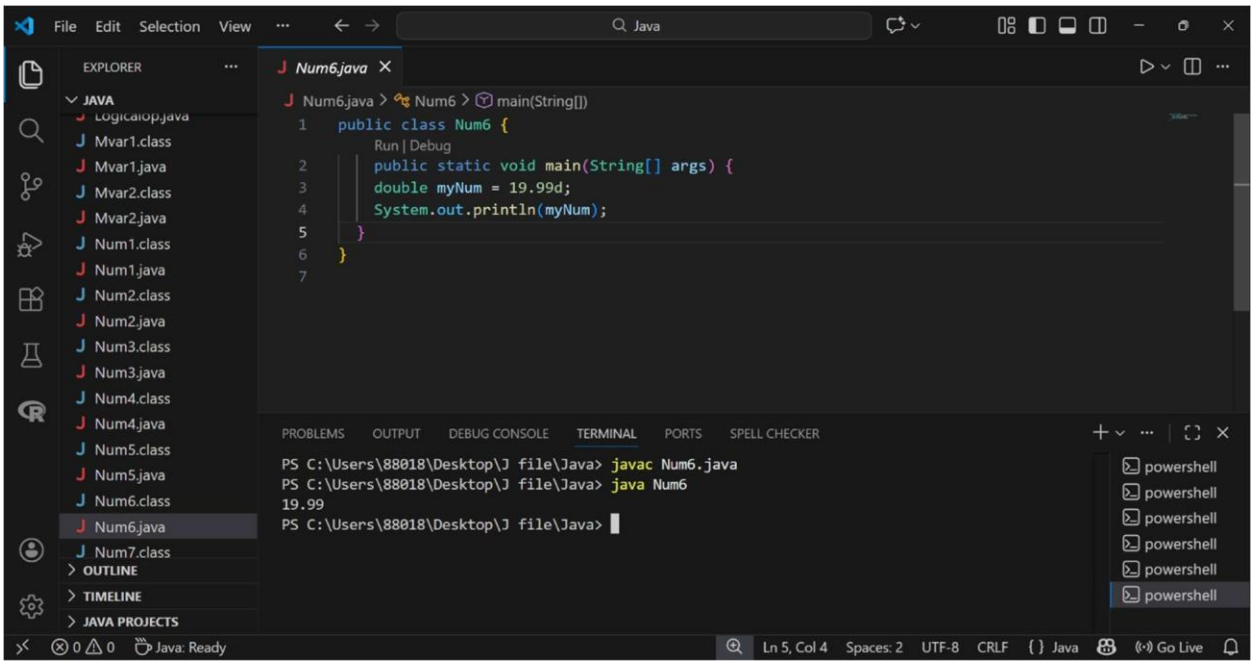
4. This Java program u lizes the long data type to store an excep onally large 64-bit integer value, indicated by the L suffix, and prints it to the console.



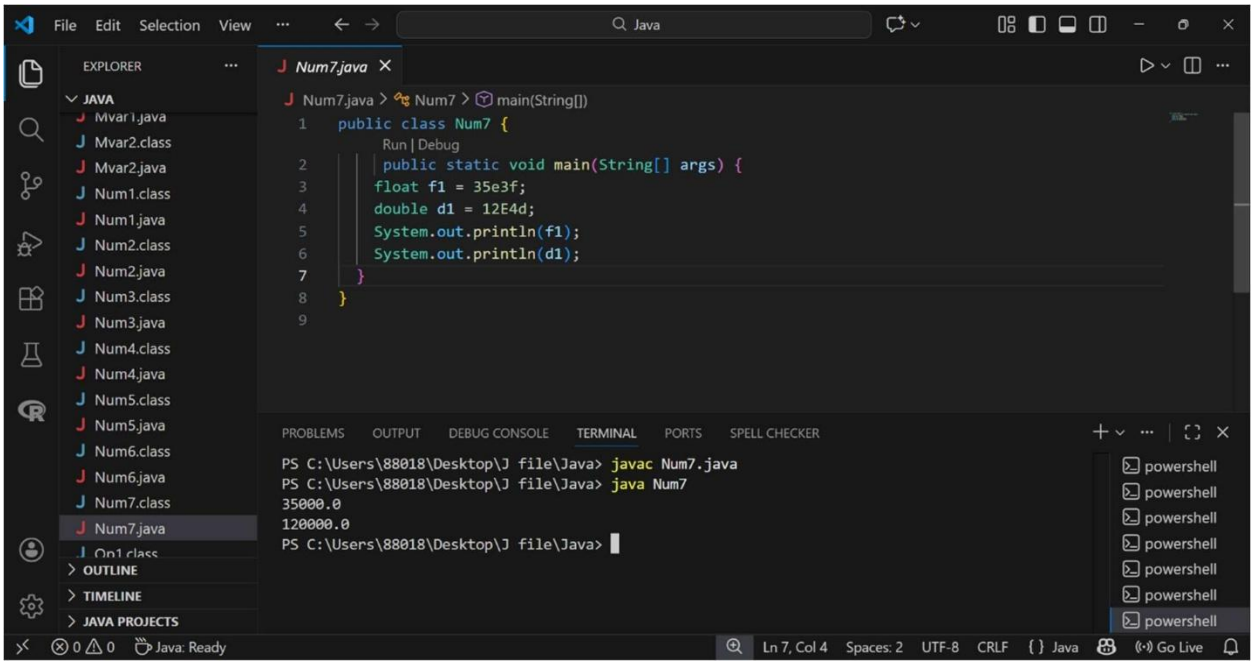
5. This program demonstrates the float data type for storing single-precision decimal numbers, iden fied by the required f suffix.



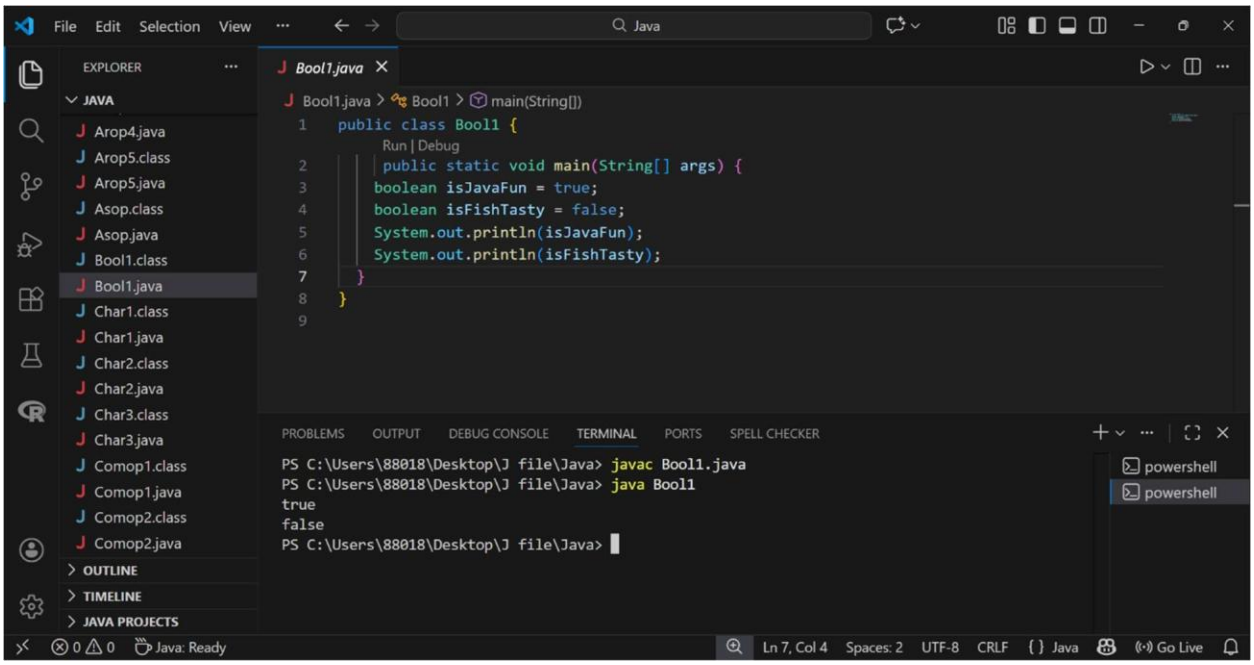
6. This code u lizes the double data type to store a double-precision floa ng-point number, offering higher precision for decimal values.



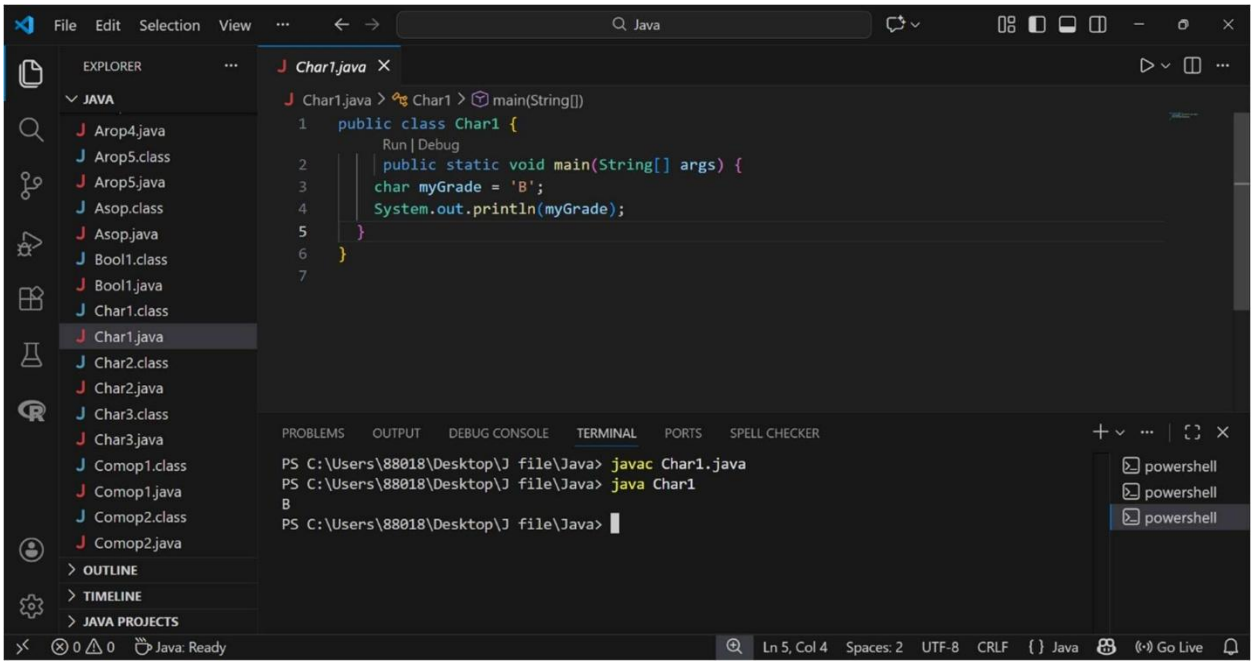
7. Features the use of scien fic nota on (e.g., 35e3f) to assign powers-of-ten values to floa ng-point and double variables.



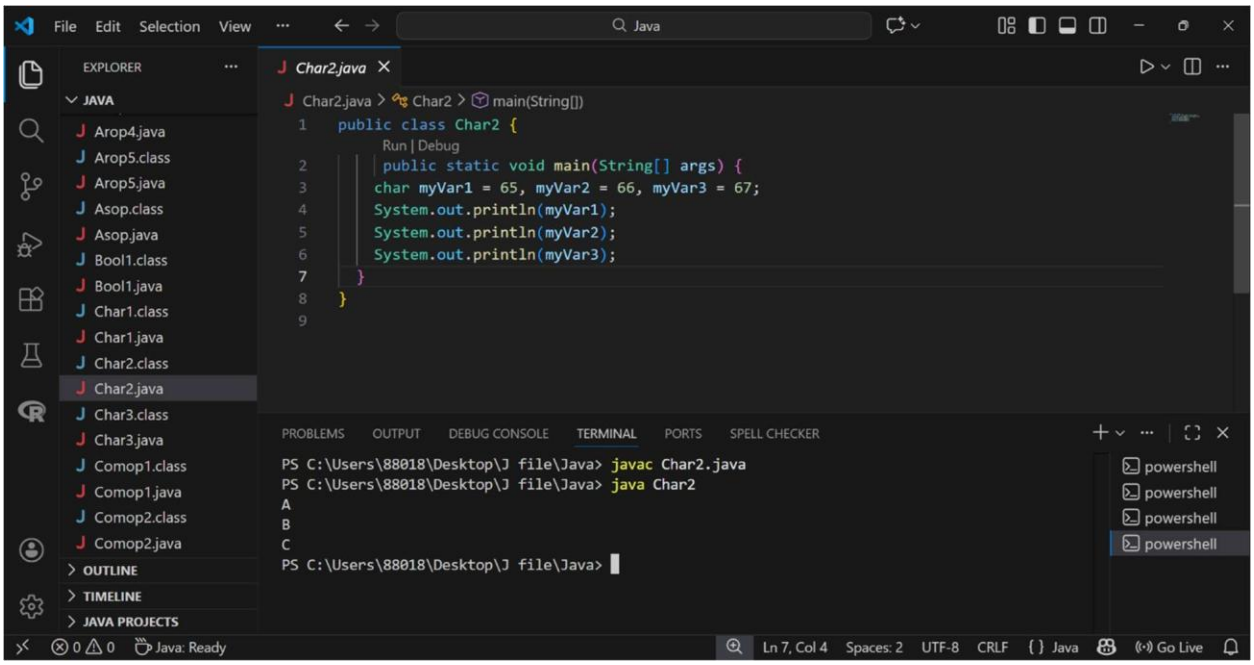
8. Introduces the boolean data type, which is used to store logical values represen ng only true or false states.



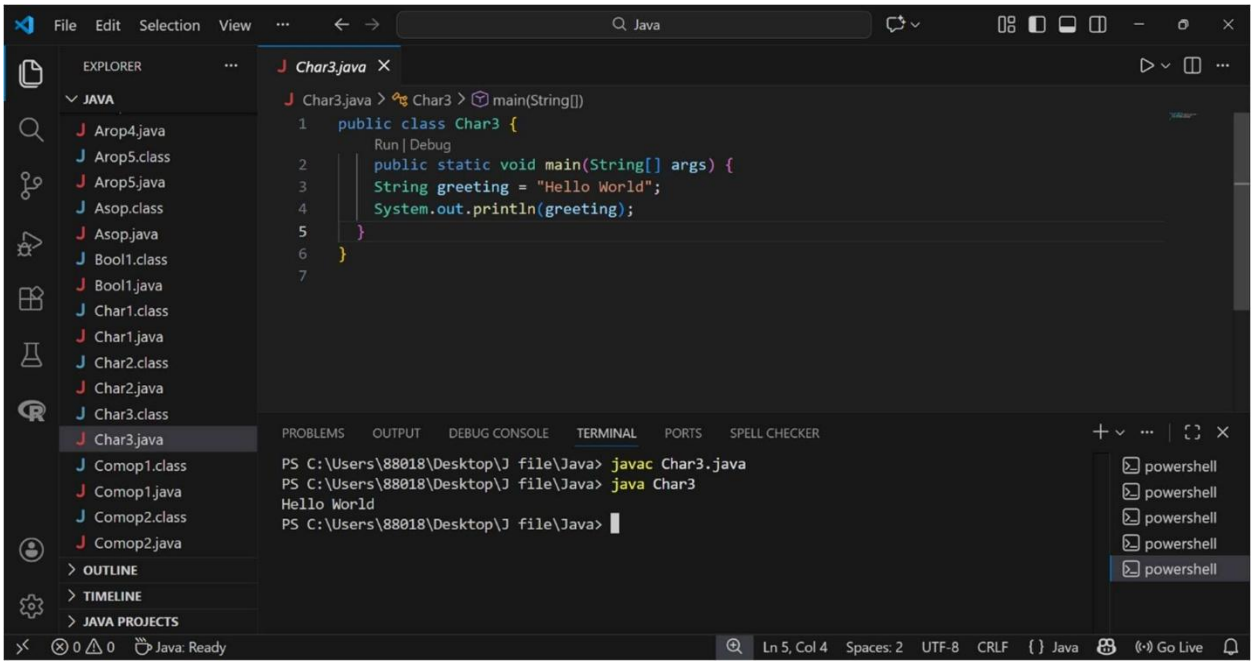
9. Demonstrates the char data type used to store a single character, enclosed in single quotes.



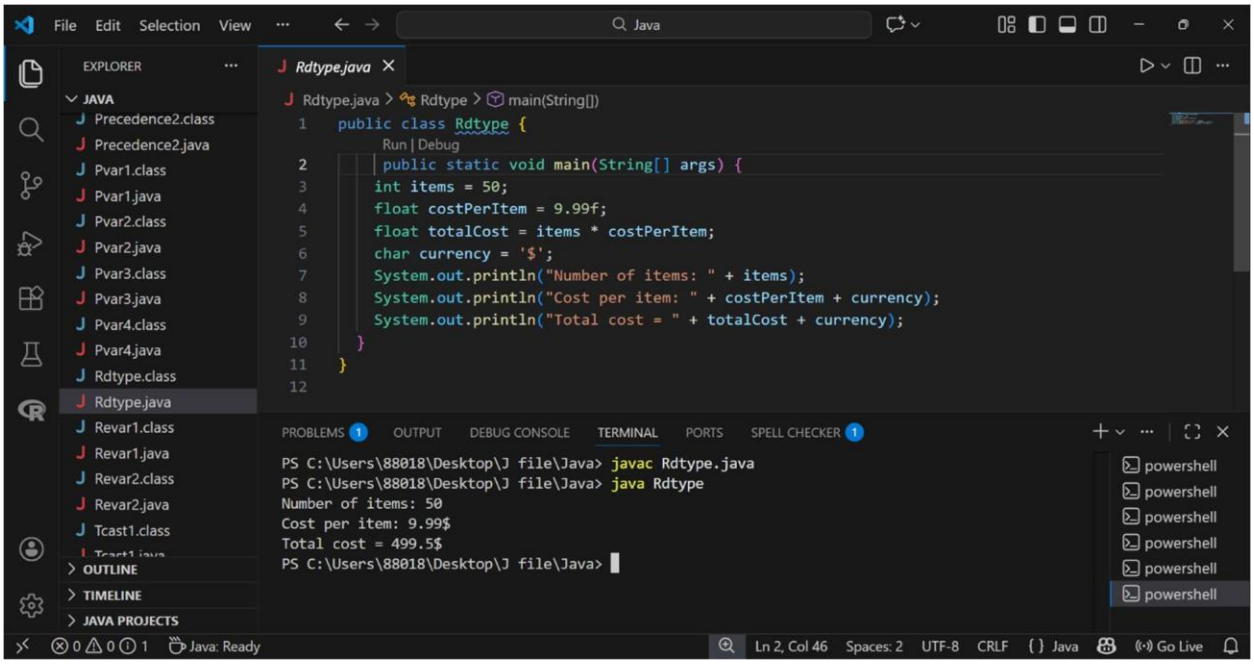
10. Illustrates how ASCII values can be assigned to char variables to display their corresponding characters.



11. Introduces the String reference data type, which is used to store and display sequences of characters or text.

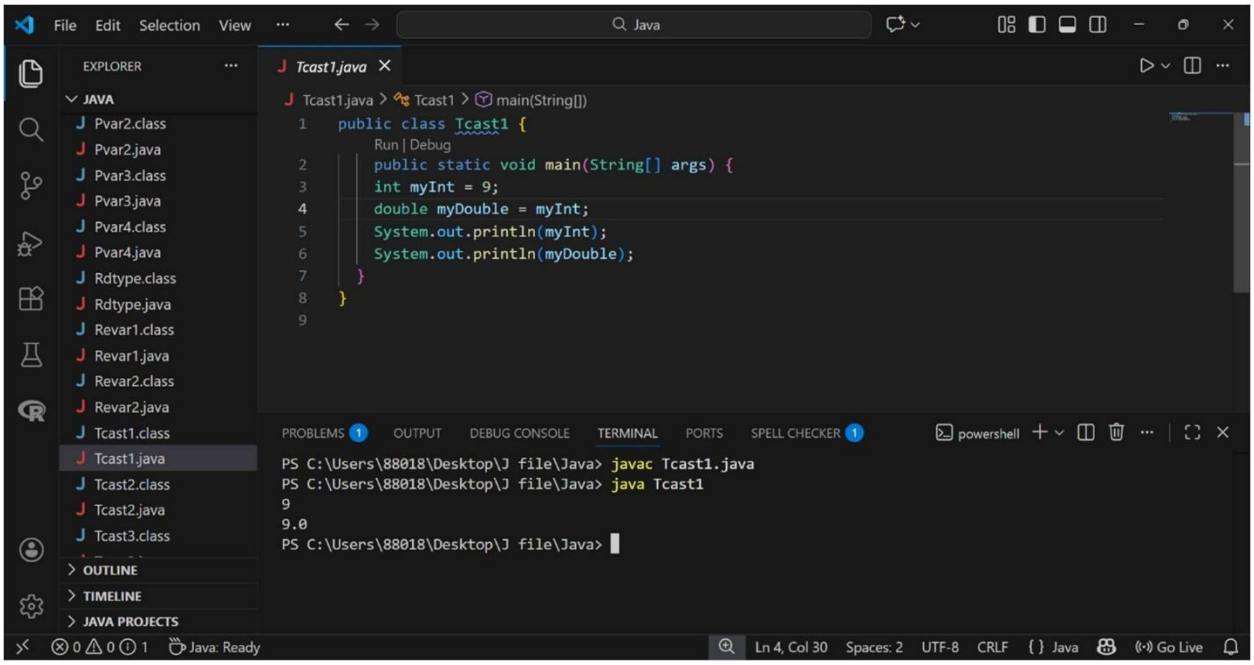


12. Demonstrates a prac cal applica on by combining mul ple data types (int, float, char) to calculate and format a total cost output.

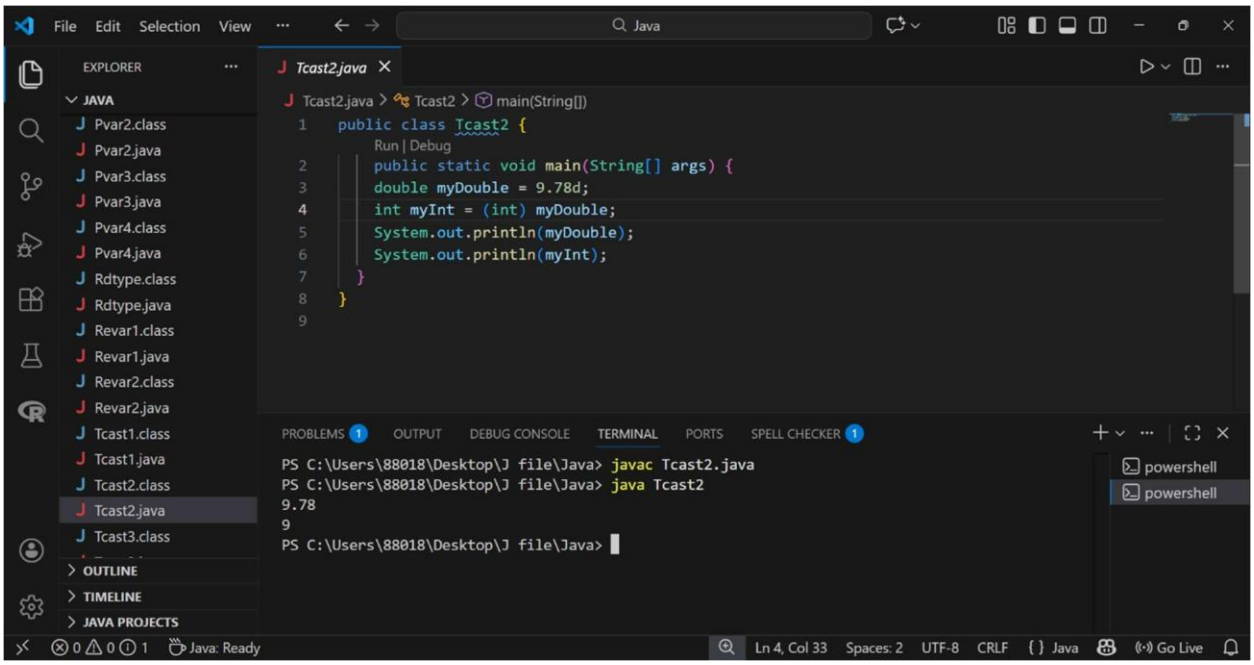


TypeCas ng sec on:

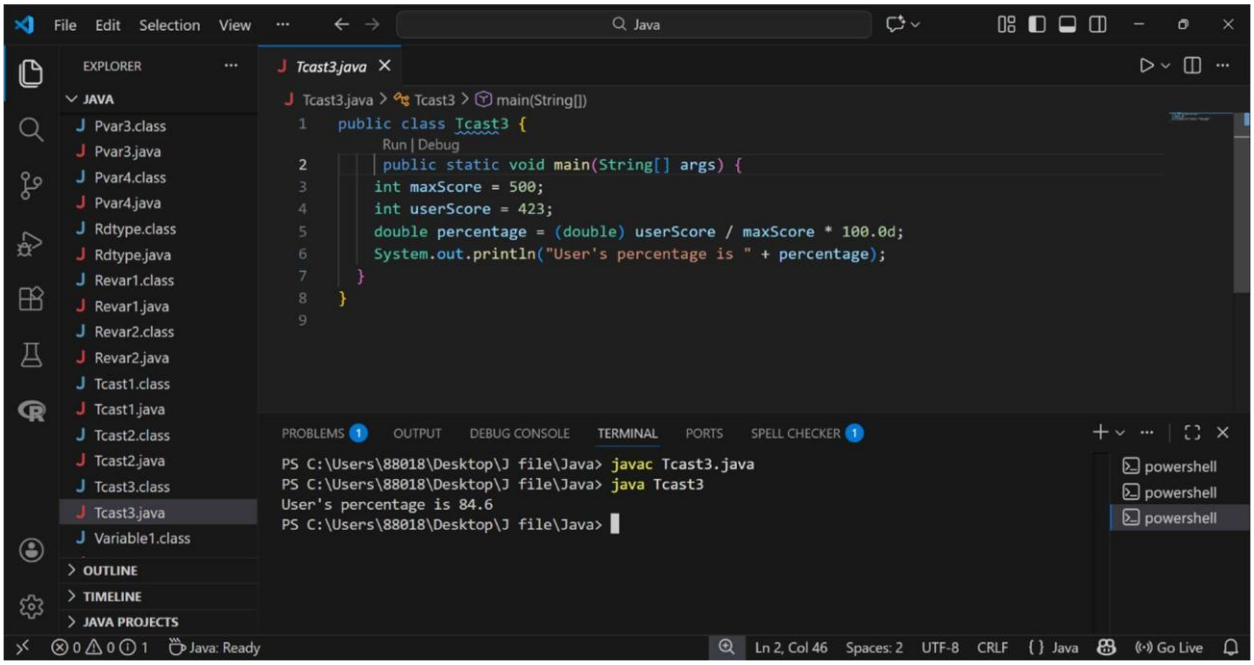
1. Illustrates Widening Cas ng, where a smaller type (int) is automa cally converted to a larger type (double) by the Java compiler.



2. Showcases narrowing cas ng, where a double is manually converted to a smaller int type, resul ng in decimal trunca on.

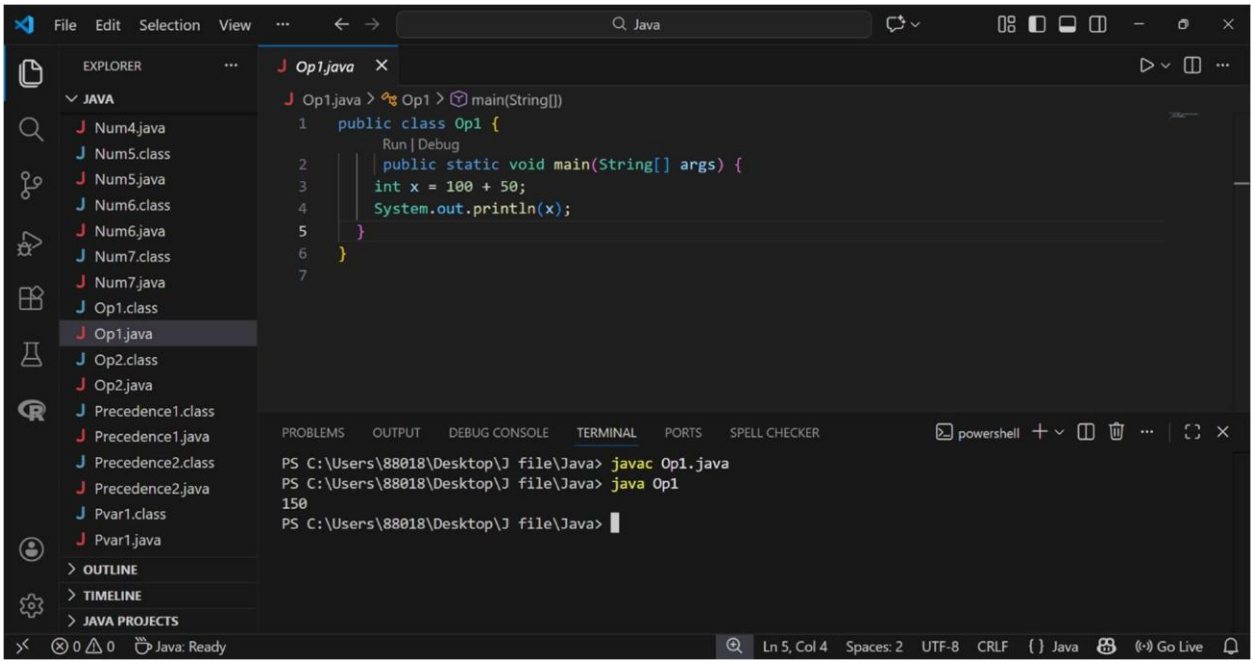


3. Demonstrates a real-world use of type cas ng to perform floa ng-point division between two integers to calculate a percentage.

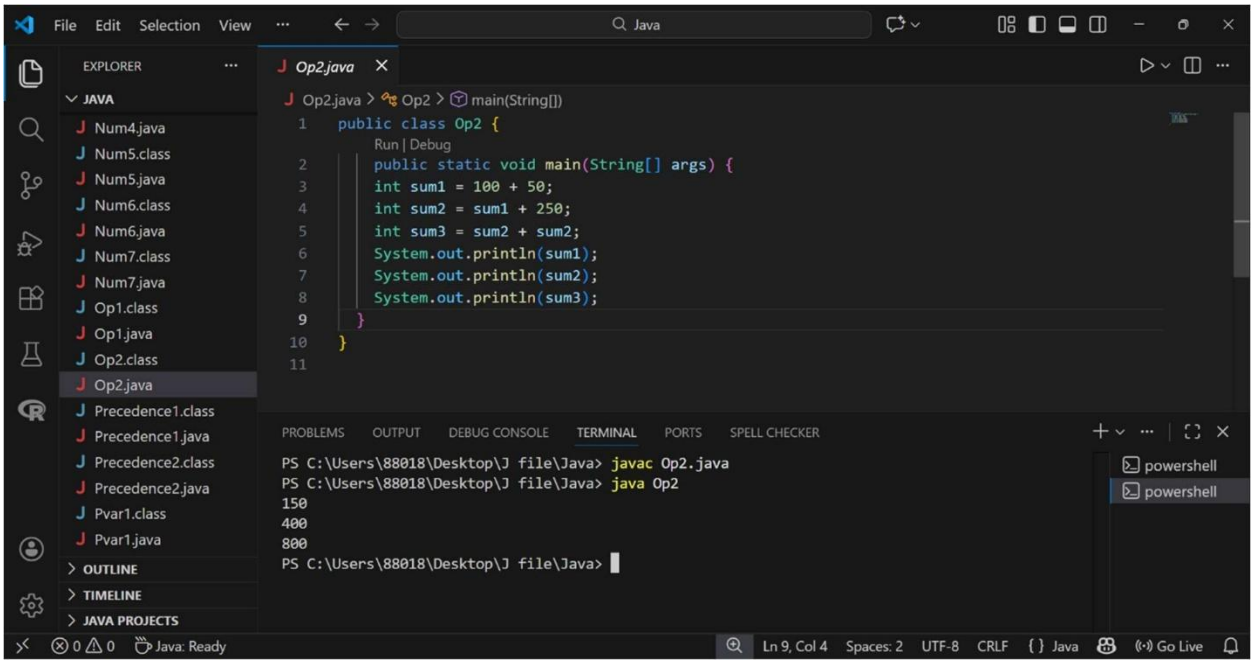


Opareators:

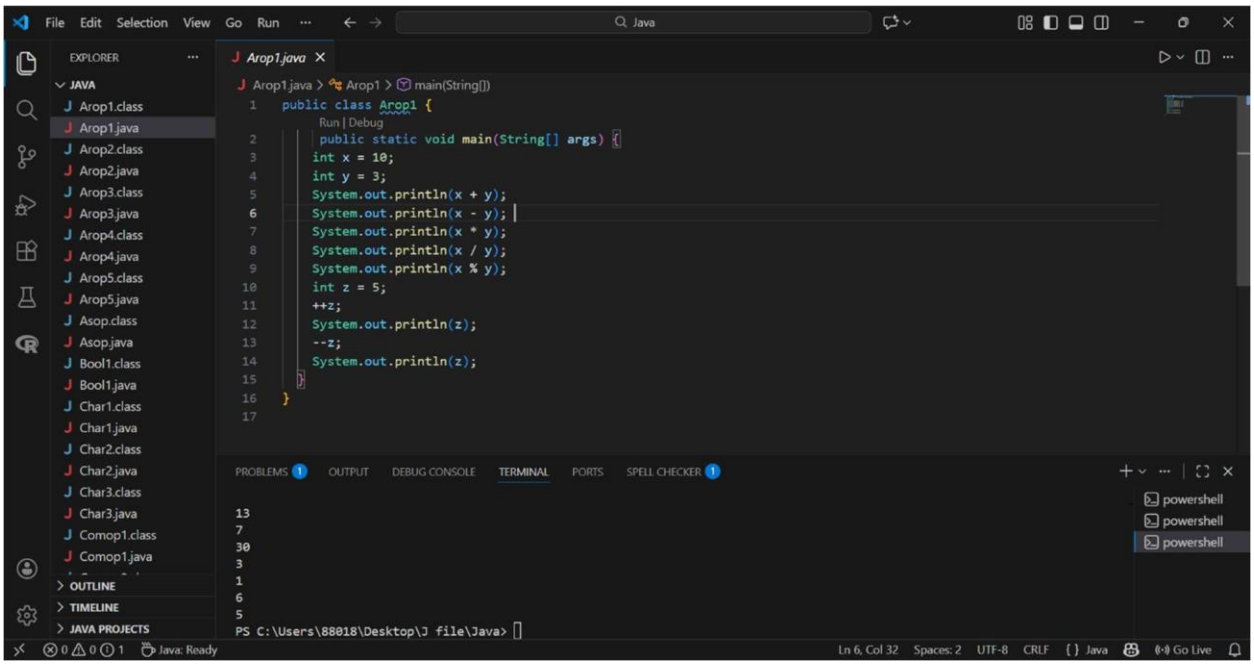
1. The provided Java code demonstrates a basic arithme c opera on where the integer variable x is assigned the sum of 100 and 50, and the resul ng value, 150, is displayed in the console using the System.out.println() method.



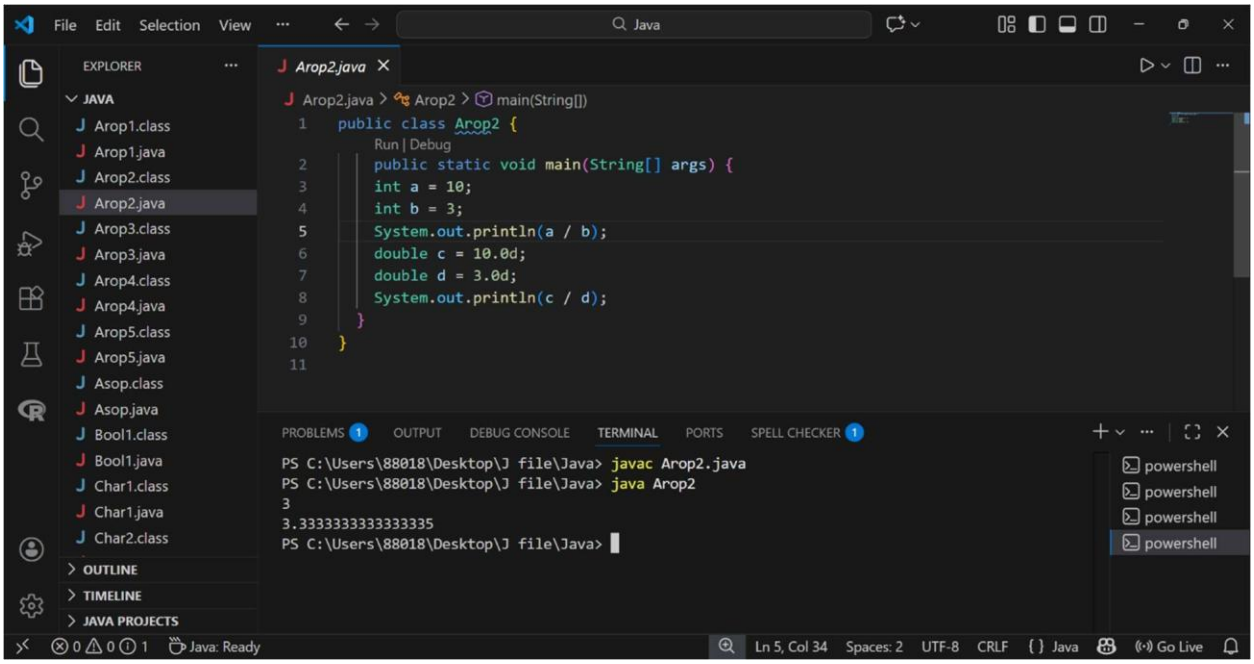
2. This Java program illustrates the sequen al use of the addi on operator to perform calcula ons using both literal values and previously defined variables, outpu ng the results of each step to the terminal.



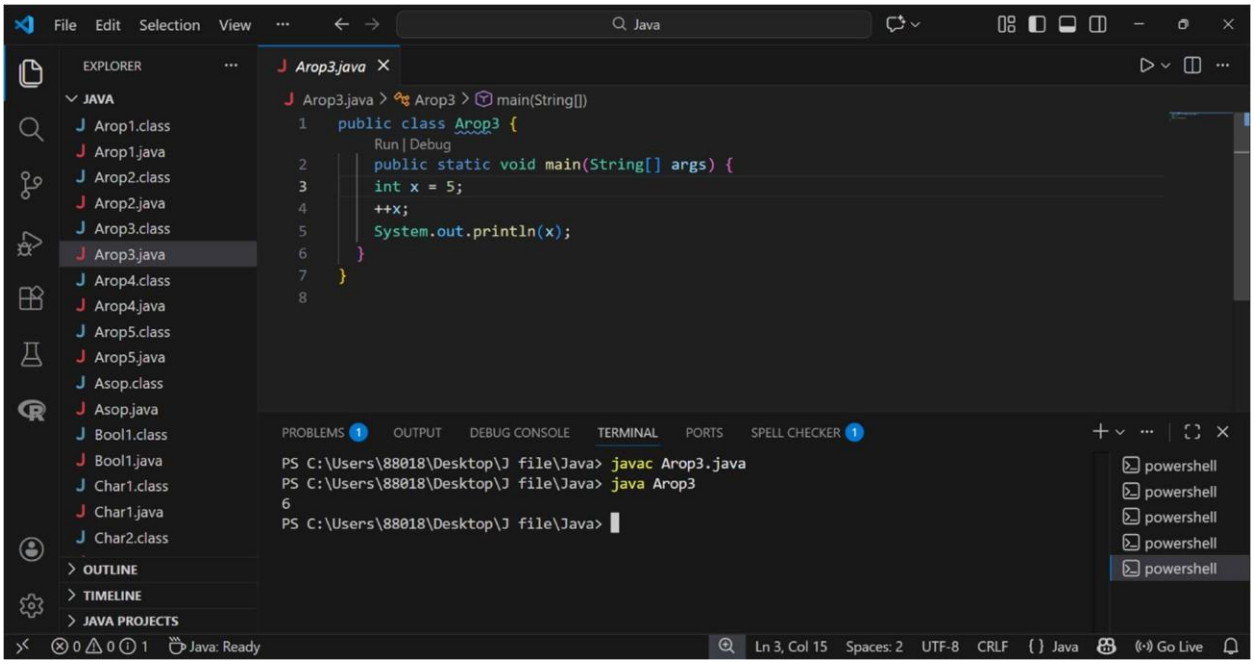
3. This Java program demonstrates the applica on of basic arithme c operators (addi on, subtrac on, mul plica on, division, and modulo) alongside prefix increment and decrement operators to manipulate and output integer values.



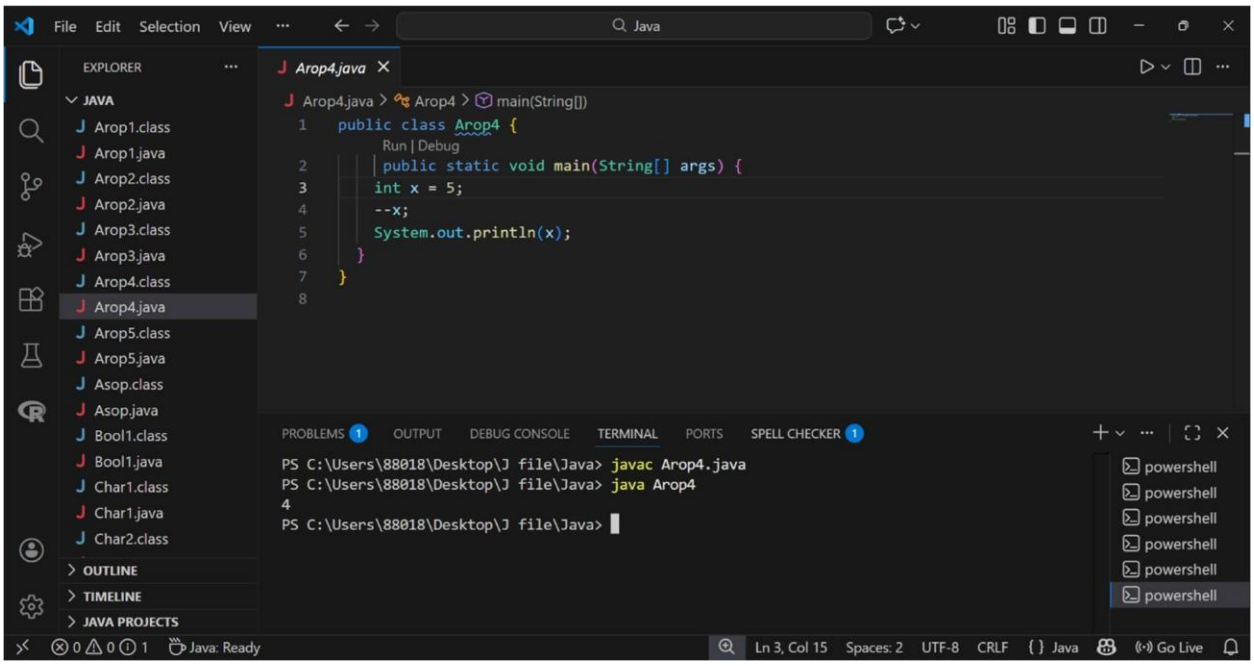
4. This Java program highlights the difference between integer and floa ng-point division by demonstra ng how the former truncates decimal values while the double type preserves precision during calcula ons.



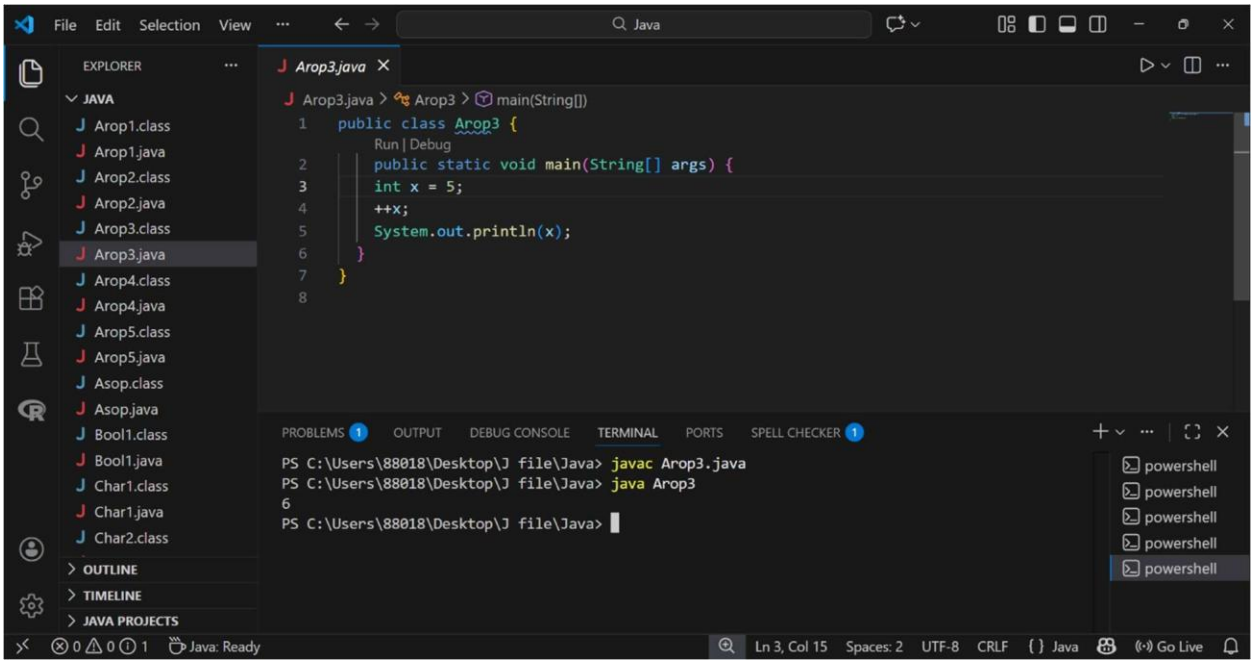
5. This Java program demonstrates the use of the prefix increment operator (++x) to increase an integer variable's value by one before it is processed and displayed in the console output.



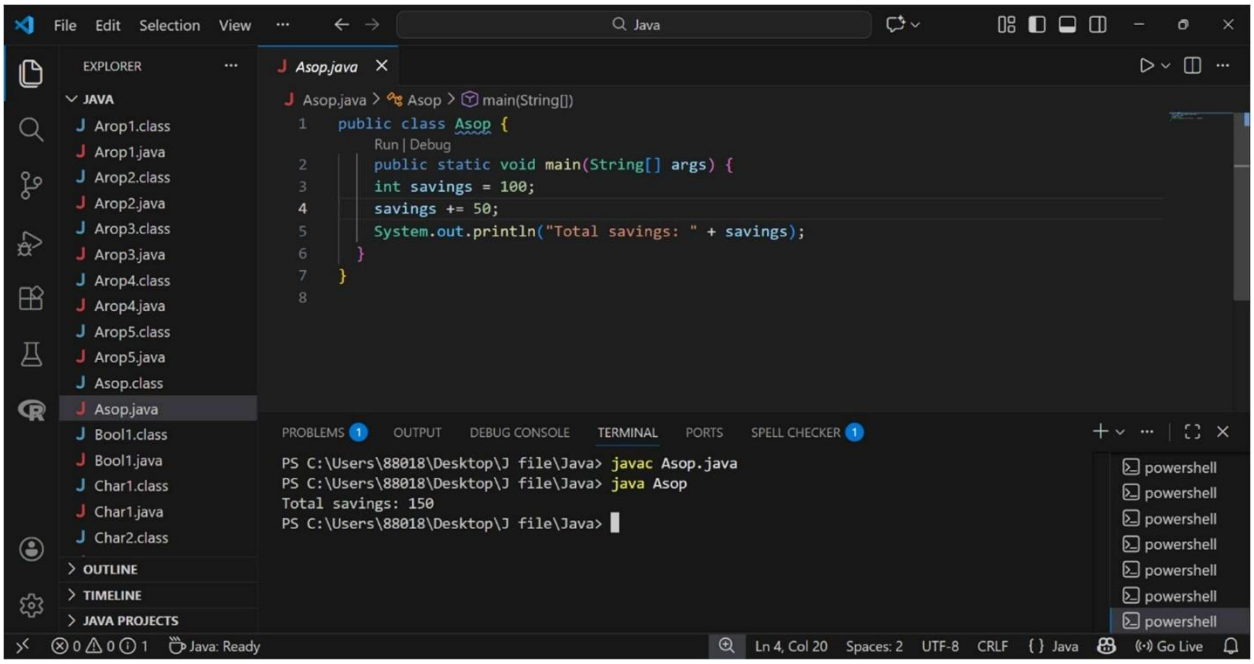
6. This Java program u lizes the prefix decrement operator (--x) to decrease the value of an integer variable by one before prin ng the updated result to the console.



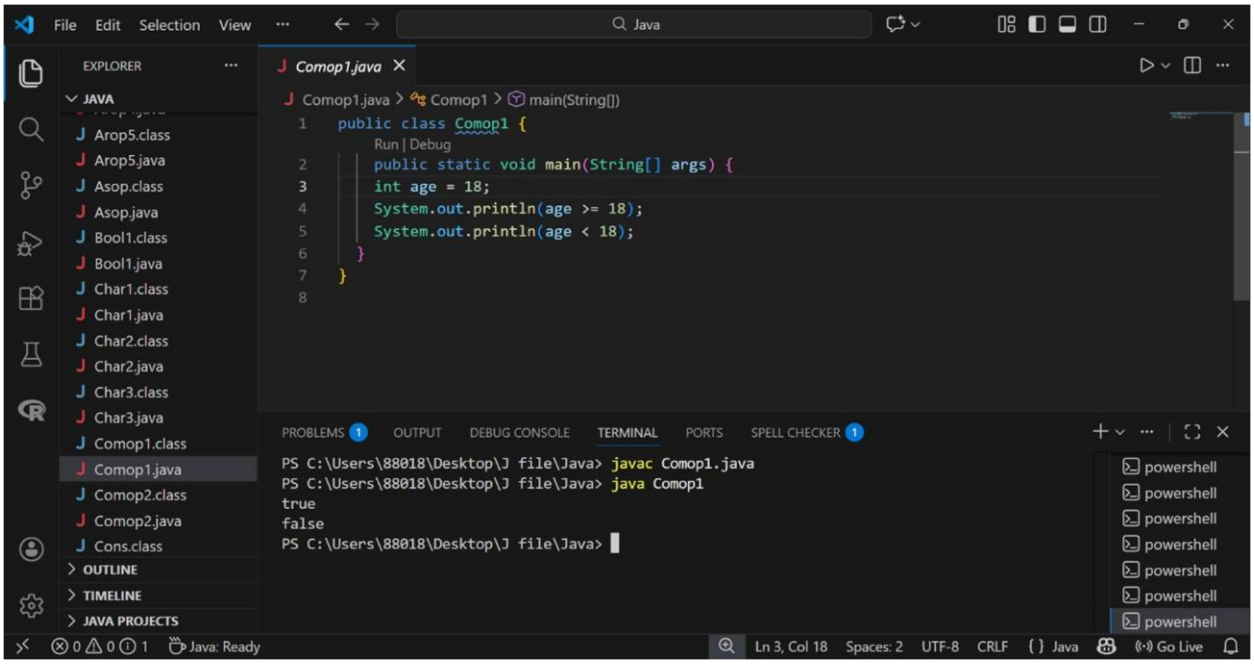
7. This Java program demonstrates the use of pos ix increment (++) and pos ix decrement (--) operators to itera vely modify a variable's value, simula ng a simple counter for tracking items or individuals.



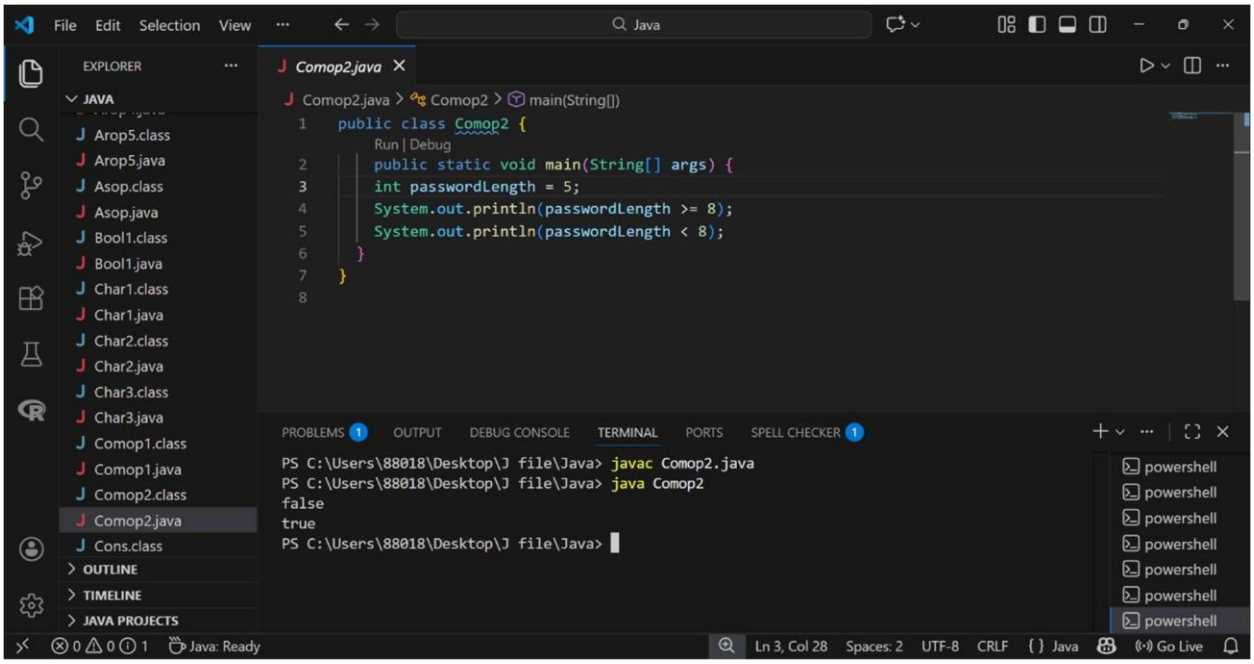
8. This Java program demonstrates the use of the addi on assignment operator (+=) to update a variable's value by adding a specific amount to its exis ng total and displaying the result.



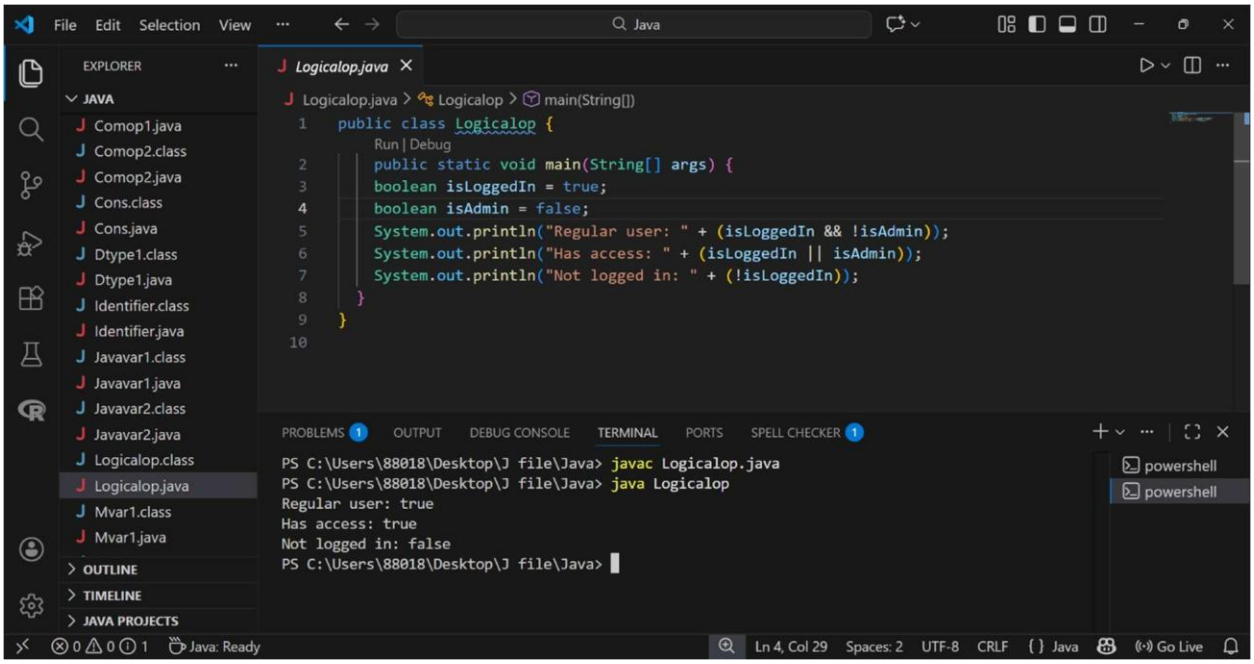
9. This Java program demonstrates the use of comparison operators, specifically "greater than or equal to" (>=) and "less than" (<), to evaluate a numerical condi on and return a boolean true or false result.



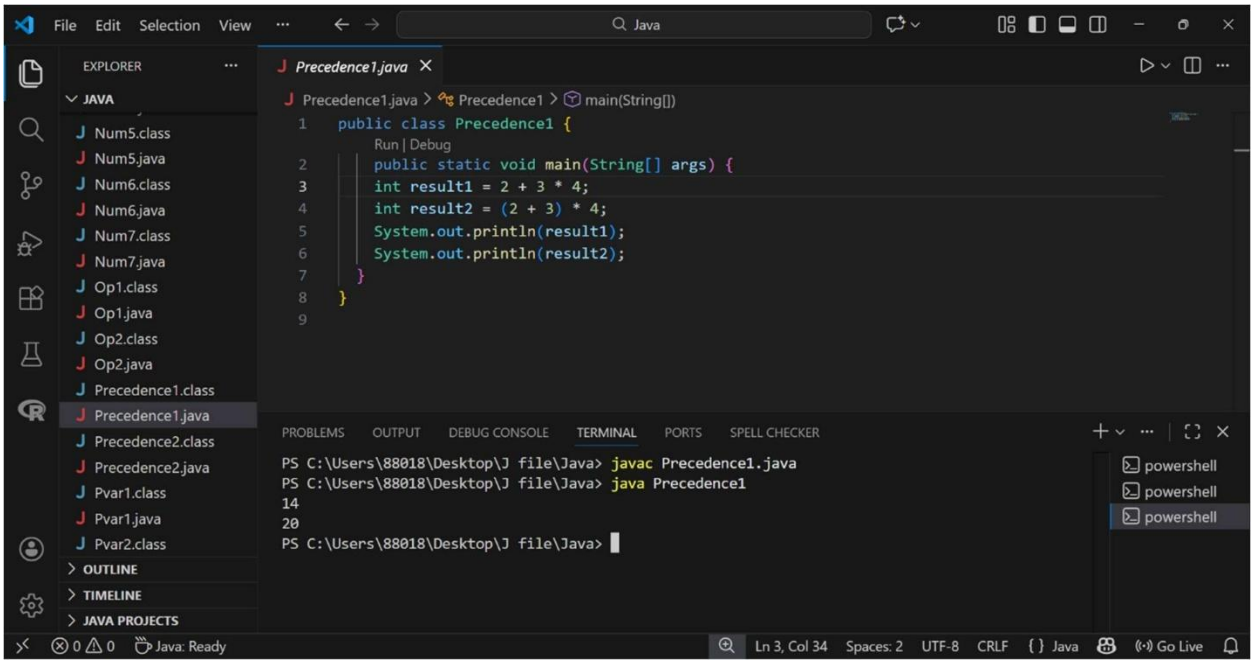
10. This Java program u lizes comparison operators ($>=$ and $<$) to evaluate if a variable represen ng password length meets a minimum character requirement, outpu ng the boolean results to the console.



11. This Java program demonstrates the applica on of logical operators, specifically AND (&&), OR (||), and NOT (!), to evaluate boolean variables and determine condi onal outcomes based on user login and administrator status.



12. This Java program illustrates operator precedence by showing how mul plica on is priori zed over addi on by default, and how parentheses can be used to override this order to perform addi on first.



13. This Java program demonstrates operator associa vity and precedence by showing how subtrac on and addi on are evaluated from le to right by default, and how parentheses can be used to group opera ons to change the final result.